

MPC-PiNNs technique applied to Quadrotors

Elective in Robotics [UAV] Course Project

Master's Degree in Artificial Intelligence and Robotics

Authors (matricole)

Academic Year 2024/2025



SAPIENZA
UNIVERSITÀ DI ROMA



Table of Contents

1 Introduction

- ▶ Introduction
- ▶ Quadrotor Model
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered



Neural Model Predictive Control (NMPC)

1 Introduction

In this work we have tried to combine Model Predictive Control along with PiNNs to trajectory tracking problem applied to quadrotors. We have used the Unicycle robot as simple test case for our experiments.



Table of Contents

2 Quadrotor Model

- ▶ Introduction
- ▶ **Quadrotor Model**
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered



Quadrotor Standard Model

2 Quadrotor Model

Starting from the Translational (1) and Rotational Dynamics (2), we have derived the simplified second order of the quadrotor.

$$m\ddot{\mathbf{p}} = \mathbf{f}_t + \mathbf{f}_g + \mathbf{f}_a + \mathbf{f}_d \quad (1)$$

$$\mathbf{J} \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J} \boldsymbol{\omega}) = \boldsymbol{\tau} \quad (2)$$

$$\begin{cases} \ddot{x} = -\left(\cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi\right) \frac{T}{m}, \\ \ddot{y} = -\left(\sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi\right) \frac{T}{m}, \\ \ddot{z} = -\left(\cos \theta \cos \phi\right) \frac{T}{m} + g, \\ \ddot{\phi} = \frac{\tau_{\phi}}{I_x} \\ \ddot{\theta} = \frac{\tau_{\theta}}{I_y} \\ \ddot{\psi} = \frac{\tau_{\psi}}{I_z} \end{cases}$$

(3)



Crazyflie Model

2 Quadrotor Model

Since we have used the Crazyflie as reference, the reference model is different with respect to the previous one. This is a consequence of how the firmware of the robot acts. There is a low level attitude controller that takes as inputs the RPY desired poses and maps them as control torques.

$$\left\{ \begin{array}{l} \dot{x} = v_x, \\ \dot{y} = v_y, \\ \dot{z} = v_z, \\ \dot{v}_x = -\left(\cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi\right) \frac{T}{m}, \\ \dot{v}_y = -\left(\sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi\right) \frac{T}{m}, \\ \dot{v}_z = -\left(\cos \theta \cos \phi\right) \frac{T}{m} + g, \\ \dot{\phi} = (\phi_c - \phi) / \tau \\ \dot{\theta} = (\theta_c - \theta) / \tau \\ \dot{\psi} = (\psi_c - \psi) / \tau \end{array} \right.$$

(4)



Table of Contents

3 Model Predictive Control

- ▶ Introduction
- ▶ Quadrotor Model
- ▶ **Model Predictive Control**
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered



MPC Structure

3 Model Predictive Control

We have used for both robots the following structure for the MPC:

$$\min_{u(0), u(1), \dots, u(N-1)} \sum_{k=0}^{N-1} \ell(x(k), u(k)) + \ell_f(x(N)) \quad (5)$$

$$\text{s.t. } x(k+1) = f(x(k), u(k)), \quad k = 0, \dots, N-1, \quad (6)$$

$$x(k) \in \mathcal{X}, \quad u(k) \in \mathcal{U}, \quad k = 0, \dots, N-1, \quad (7)$$

$$x(0) = x_0. \quad (8)$$



MPC Cost Function

3 Model Predictive Control

Cost Function

$$\ell(x(k), u(k)) = \sum_{k=0}^{N-1} \left[\sum_{x \in \mathcal{X}} W_{x(k)} \left(x(k)_{\text{ref}} - x(k) \right)^2 + \sum_{u \in \mathcal{U}} W_u u^2 \right],$$

$$\ell_f(x(N)) = \sum_{x \in \mathcal{X}} W_{x(N)} \left(x(N)_{\text{ref}} - x(N) \right)^2.$$

Unicycle State & Control

$$\begin{cases} \mathcal{X} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T, \\ \mathcal{U} = \begin{bmatrix} v & \omega \end{bmatrix}^T. \end{cases}$$

Quadrotor State & Control

$$\begin{cases} \mathcal{X} = \begin{bmatrix} x & y & z & v_x & v_y & v_z & \phi & \theta & \psi \end{bmatrix}^T, \\ \mathcal{U} = \begin{bmatrix} \phi_c & \theta_c & \psi_c & T \end{bmatrix}^T. \end{cases}$$



Table of Contents

4 Neural Network

- ▶ Introduction
- ▶ Quadrotor Model
- ▶ Model Predictive Control
- ▶ **Neural Network**
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered



Simple MLP

4 Neural Network

The architecture is characterized by two hidden layers of 128 neurons and takes as input the current state and the action that I want to apply.

$$\dot{x} = f_{\theta}(x, u) \quad (9)$$

$$loss = \frac{1}{k} \sum_{i=t}^{t+k} (\bar{s}_i - s_i)^2 \quad (10)$$

The loss is computed as the MSE between the neural and the nominal evolutions of the state.

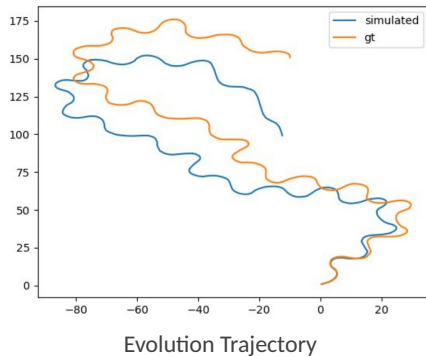
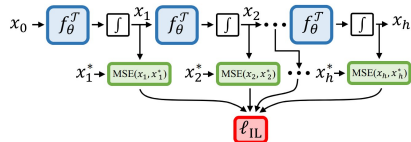




Table of Contents

5 Physical Informed Neural Networks

- ▶ Introduction
- ▶ Quadrotor Model
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ **Physical Informed Neural Networks**
- ▶ Results and Problems Encountered



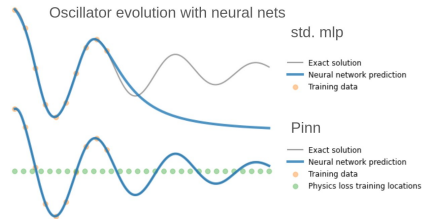
PiNNs

5 Physical Informed Neural Networks

Physical Informed Neural Networks (PiNNs) propose an alternative approach where the solution is approximated by a neural network that is trained not only on data but also by enforcing the underlying physical laws via the differential equations.

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_{\theta}(\mathbf{x}_i, t_i) - u_i^{\text{obs}}|^2, \quad (11)$$

$$\mathcal{L}_{\text{phys}}(\theta) = \frac{1}{N_r} \sum_{j=1}^{N_r} |\mathcal{N}[u_{\theta}](\mathbf{x}_j, t_j)|^2, \quad (12)$$



PiNN vs Classical MLP

Problems and Limitations:

- Cannot handle control input
- Fixed initial condition



PiNN with Controls

5 Physical Informed Neural Networks

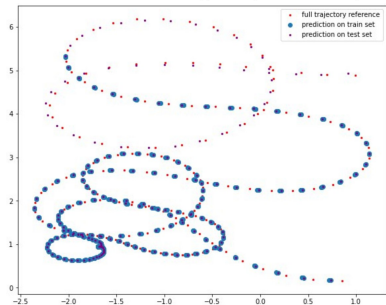
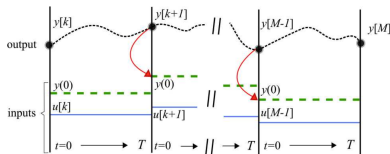
The Pinn-C modifies the formulation of the pinn, by dividing the evolution into sub-transitions with constant input. The network takes as input $(x(o), \bar{u}, t)$.

Pros:

- it can handle larger time-steps with respect to the original formulation
- can accept arbitrary starting conditions

Cons:

- relies heavily on the dataset



Trajectory with PiNN-C



Table of Contents

6 Results and Problems Encountered

- ▶ Introduction
- ▶ Quadrotor Model
- ▶ Model Predictive Control
- ▶ Neural Network
- ▶ Physical Informed Neural Networks
- ▶ Results and Problems Encountered



Dataset Generation

6 Results and Problems Encountered

We have tried different strategies:

- random velocity commands over the horizon
- random velocity commands constant over the horizon

The system has been integrated with Euler or Runge-Kutta 4.

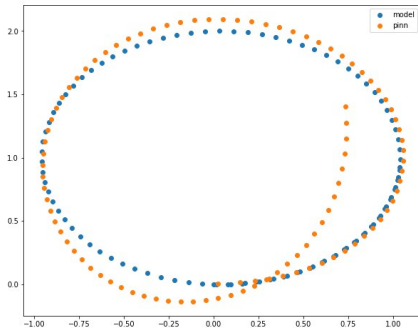


Drift Problem

6 Results and Problems Encountered

After some training, we have tested our neural model along with the MPC with nice results (check video). However, we found that by having zero control inputs the network still moves the state somewhere else. We have tried to overcome this problem in some ways but with poor results:

- Sample N points at the boundary during each epoch
- Add a penalty term related to the boundary condition



PiNN-C vs Reference



Possible Local Minima

6 Results and Problems Encountered

During each training experiment, we have found that all losses behave correctly but they remain stuck at some points even with few iterations. Also in this case we have tried extensively hyperparameter tuning without any solution to the problem.

